



南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

本科生毕业设计（论文）

题 目：大模型赋能的中小学教育个性化答疑系统

姓 名：高进

学 号：12212618

院 系：计算机科学与工程

专 业：计算机科学与技术

指导教师：张殷乾

2026 年 5 月 6 日

诚信承诺

1. 本人郑重承诺所呈交的毕业设计（论文），是在导师的指导下，独立进行研究工作所取得的成果，所有数据、图片资料均真实可靠。
2. 除文中已经注明引用的内容外，本论文不包含任何其他人或集体已经发表或撰写过的作品或成果。对本论文的研究作出重要贡献的个人和集体，均已在文中以明确的方式标明。
3. 本人承诺在毕业论文（设计）选题和研究内容过程中没有抄袭他人研究成果和伪造相关数据等行为。
4. 在毕业论文（设计）中对侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

作者签名：

2026 年 5 月 6 日

大模型赋能的教育个性化答疑系统

高进

(计算机科学与工程系 指导教师：张殷乾)

[摘要]：本报告阐述了基于检索增强生成（RAG）技术的教育智能助手系统的设计与实现。该系统针对中小学教育场景，构建了包含知识库管理、智能问答、文档匹配分析、自动出题、评分标准生成、代码出题与代码分析六大功能的教育智能平台。系统采用 LangChain 框架进行 RAG 流程编排，使用 Chroma 本地向量数据库存储教育资料，通过通义千问和 MiniMax 大语言模型实现智能问答服务。在知识库管理方面，系统支持 TXT、PDF 及代码文件的上传，采用 MD5 去重机制避免重复存储，使用 RecursiveCharacterTextSplitter 进行语义分割后将文本向量存储至 Chroma 数据库。在智能问答方面，系统实现了基于检索增强的问答流程，将用户查询与检索到的上下文及对话历史整合后输入大语言模型生成回答，并支持流式输出。在扩展功能方面，系统提供文档匹配分析、自动出题、评分标准生成及代码出题与分析功能，服务于多元化教学场景。经过测试验证，系统各项功能均能正常工作：知识库上传成功处理多种格式文件；智能问答能正确检索相关文档并生成准确答案；对话历史管理支持多轮对话；模型切换可在通义千问与 MiniMax 之间无缝切换。测

试结果表明，系统能够有效支持教育工作者的教学需求。

[关键词]： 检索增强生成；教育智能助手；向量数据库；LangChain；智能问答

[ABSTRACT]: This report presents the design and implementation of an educational intelligent assistant system based on Retrieval-Augmented Generation (RAG) technology. Targeting primary and secondary education scenarios, the system builds an educational intelligent platform with six core functions: knowledge base management, intelligent Q&A, document matching analysis, automatic question generation, scoring criteria generation, and code question analysis. The system uses LangChain framework for RAG workflow orchestration, Chroma local vector database for educational material storage, and Qwen and MiniMax large language models for intelligent Q&A services. In terms of knowledge base management, the system supports uploading TXT, PDF and code files, employs MD5 deduplication to prevent duplicate storage, and uses RecursiveCharacterTextSplitter for semantic segmentation. In intelligent Q&A, the system implements retrieval-augmented question-answering, integrating user queries with retrieved context and conversation history. The extended functions include document matching analysis, automatic question generation, scoring criteria generation, and code question analysis. After testing and verification, all system functions work normally. Test results

demonstrate that the system can effectively support educators' teaching needs and provide personalized educational auxiliary services.

[Key words]: Retrieval-Augmented Generation; Educational Intelligent Assistant; Vector Database; LangChain; Intelligent Q&A

目录

1. 引言.....	9
2. 基础理论及相关工作.....	11
2.1 教育智能问答系统概述.....	11
2.2 知识表示与知识图谱技术.....	13
2.3 个性化学习与自适应学习系统.....	15
2.4 国内外研究现状分析.....	16
3. 关键技术综述.....	19
3.1 检索增强生成技术.....	19
3.2 向量数据库技术.....	19
3.3 LangChain 框架.....	20
3.4 Streamlit Web 框架.....	20
4. 需求分析与可行性研究.....	21
4.1 用户需求分析.....	21
4.2 技术可行性分析.....	22
4.3 经济可行性分析.....	22
5. 系统设计.....	23
5.1 设计难点与技术挑战.....	23
5.2 总体架构设计.....	24
5.3 知识库管理模块.....	24
5.4 向量检索模块.....	26
5.5 智能问答模块.....	28

5.6 扩展功能模块.....	28
5.7 技术选型.....	29
5.8 验证指标.....	29
6. 系统实现.....	30
6.1 开发环境与工具.....	30
6.2 知识库服务实现.....	30
6.3 向量检索服务实现.....	31
6.4 RAG 服务实现.....	31
6.5 模型工厂实现.....	31
6.6 Web 界面实现.....	31
7. 系统测试.....	32
7.1 测试环境与方法.....	32
7.2 功能测试结果.....	32
7.3 性能测试结果.....	32
7.4 用户体验测试.....	33
7.5 问答质量评估.....	33
7.6 效率分析.....	34
7.7 不确定性与系统稳定性分析.....	35
7.8 算法效率分析.....	36
8. 总结与展望.....	37
8.1 研究工作总结.....	37
8.2 存在问题与改进方向.....	38

8.3 未来工作展望.....	38
9. 参考文献.....	40
10. 附录：核心代码.....	42
11. 致谢.....	46

1. 引言

随着人工智能技术的飞速发展，特别是大语言模型（Large Language Models, LLMs）技术的突破，人工智能在教育领域的应用前景日益广阔。从早期的专家系统到如今的深度学习模型，人工智能技术一直在教育领域扮演着重要的角色。然而，传统的智能问答系统往往面临着知识更新困难、回答缺乏事实依据、无法理解上下文语义等问题，严重制约了其在教育场景中的应用效果。具体而言，传统问答系统主要依赖预先构建的知识库或规则匹配机制，当学生提出的问题超出预设范围时，系统往往无法提供有效的回答；同时，由于缺乏深度语义理解能力，传统系统难以准确把握问题的核心意图，容易产生答非所问的情况。此外，传统系统的知识库更新需要专业人员介入，实时性差，难以跟上知识迭代的速度。

检索增强生成（Retrieval-Augmented Generation, RAG）技术的出现为解决上述问题提供了有效的技术方案。系统选择 RAG 作为解决方案[18]，通过结合外部知识库检索与生成模型[20]，既能发挥模型强大的语言理解和生成能力，又能确保回答基于真实可靠的知识来源，显著降低了模型产生“幻觉”问题的风险，提高了回答的准确性和可信度。2020 年，Facebook AI 研究团队提出了 RAG 技术架构[1]，该架构通过将信息检索系统与语言模型生成能力相结合，开创了知识密集型自然语言处理任务的新范式。在 RAG 框架下，当用户提出问题时，系统首先从外部知识库中检索相关文档，然后将这些文档作为上下文提供给语言模型，从而生成更加准确和可靠的回答。这种方法使得模型能够“看到”最新的、特定领域的知识，而不仅仅依赖于训练数据中的静态信息。

在中小学教育场景中，教师和学生对于智能化教育辅助工具的需求日益迫切。一方面，教师需要花费大量时间批改作业、回答学生的基础问题，难以将精力集中在教学设计和个性化指导上。根据相关调查数据显示，中小学教师每天花费在答疑和作业批改上的时间平均达到 2-3 小时，这严重挤压了教师用于教学研究和个性化辅导的时间。传统的人工答疑方式效率低下，且难以保证回答的一致性和准确性。另一方面，学生在学习过程中遇到问题时，往往难以及时获得准确的解答和指导，影响学习效率和积极性。特别是在课后自主学习阶段，学生缺乏即时的专业指导，遇到疑难问题时只能等到下次上课时询问教师，这种延迟不仅降低了学习效率，还可能导致学生对问

题的遗忘，削弱学习效果。因此，开发一个能够为中小学教育场景提供智能化辅助的系统具有重要的实际应用价值，体育教学辅助领域已有成功应用案例[7]。

从技术发展角度来看，当前 RAG 技术虽然已在金融、医疗、法律等领域得到广泛应用，但在教育领域的应用仍处于探索阶段，存在诸多挑战需要解决。首先，教育领域的知识具有高度的体系性和层次性，知识点之间存在复杂的关联和依赖关系，简单的文档检索难以捕捉这种结构化的知识组织方式。其次，教育场景对回答的准确性和可靠性要求极高，尤其是涉及学科概念、公式应用、解题方法等内容时，任何偏差都可能对学生造成误导。第三，不同学科、不同年级的学生具有差异化的学习需求，统一的问答系统难以满足这种个性化的教育需求。第四，学生的提问方式往往不够规范，存在大量的口语化表达、缩写、别称等，这给问题的理解和知识检索带来了挑战。针对这些挑战，研究人员开始探索将 RAG 技术与教育领域特点相结合的新方法，以期构建更加适合教育场景的智能问答系统。

在国内外研究现状方面，已有多个研究团队开展了相关探索。在国外，斯坦福大学的研究团队提出了基于 RAG 的智能 tutoring 系统框架，通过结合教材内容和学生历史问答记录，为学生提供个性化的学习辅导；麻省理工学院的研究者开发了针对理工科教育的智能答疑系统，该系统能够理解学生的问题意图并提供详细的解题步骤；谷歌 DeepMind 则探索了将知识图谱与 RAG 相结合的方法，以提高检索的准确性和可解释性。在国内，北京师范大学、华东师范大学等高校的研究团队也在积极探索人工智能技术在教育领域的应用，提出了一系列面向 K12 教育的智能问答系统原型，智慧燃气领域也有应用案例[11]，JavaScript 教学辅助智能体[12]取得了良好效果，同时在心理健康教育领域，智能桌宠系统[13]证明了该技术的有效性，以及高校财务问答系统[14]、Python 知识问答系统[15]、卫星领域，张衡一号 RAG 问答系统[16]、智慧建筑客服系统[17]、人才政策智能问答系统[19]。然而，现有的教育问答系统仍存在以下不足：一是多数系统仅支持简单的问答功能，缺乏教学辅助功能如自动出题、评分标准生成等；二是系统的检索策略未能充分考虑教育知识的结构化特点，检索结果往往孤立分散；三是缺乏对多轮对话的支持，难以实现连贯的教学互动。

针对上述问题与不足，本项目旨在构建一个基于 RAG 技术的教育智能助手系统，专门服务于中小学教育场景。该系统不仅可以进行智能问答，为学生提供即时的学习辅导，还能支持文档分析、自动出题、评分标准生成等多样化的教学辅助功能，为教

师和学生提供全面的智能化支持。通过该系统,教师可以更高效地完成教学辅助工作,学生可以获得更加个性化的学习支持,整个教育过程将变得更加智能化和高效化。本系统的核心创新点包括:第一,设计并实现了窗口检索策略,通过两阶段检索方法有效解决了跨片段语义割裂问题,提升了检索质量和问答准确性;第二,构建了多链并行的 RAG 架构,支持知识问答、文档匹配分析、自动出题、评分标准生成、代码出题与代码分析等多种教育辅助功能;第三,实现了模型无关的架构设计,支持通义千问和 MiniMax 等多种大语言模型的无缝切换,提高了系统的适用性和灵活性。

本论文将从关键技术综述、系统需求分析、系统设计、系统实现、系统测试等多个方面,对该教育智能助手系统进行全面阐述。论文的主要贡献包括:第一,设计并实现了一个基于 RAG 技术的教育智能助手系统,系统功能完善、性能稳定;第二,提出了窗口检索策略,有效解决了跨片段语义割裂问题;第三,开发了多种教育辅助功能,包括自动出题、评分标准生成等,丰富了系统的应用场景;第四,构建了模块化的系统架构,便于后续的功能扩展和性能优化。本论文的章节安排如下:第一章为引言,介绍研究背景、问题与挑战、国内外研究现状以及本文的主要贡献;第二章为基础理论及相关工作,介绍教育智能问答系统的定义与发展、知识表示与知识图谱技术、个性化学习理论及国内外研究现状分析;第三章为关键技术综述,介绍 RAG 技术、向量数据库、LangChain 框架和 Streamlit Web 框架的核心原理;第四章为需求分析与可行性研究,详细分析用户需求、技术可行性和经济可行性;第五章为系统设计,阐述系统的总体架构和各个功能模块的设计;第六章为系统实现,描述系统的开发环境、核心代码实现和关键算法;第七章为系统测试,展示系统的功能测试结果、性能测试结果、用户体验测试结果和算法效率分析;第八章为总结与展望,总结本文的工作成果并展望未来的研究方向。

通过本项目的研究与实践,我们期望为 RAG 技术在教育领域的应用提供有益的探索 and 参考。随着大语言模型技术的不断进步和教育场景需求的持续深化,基于 RAG 技术的教育智能助手有望成为教师和学生的重要辅助工具,为教育信息化发展贡献力量。同时,本项目的研究成果也可为其他领域的 RAG 应用提供借鉴,促进人工智能技术的广泛应用和深入发展。

2. 基础理论及相关工作

2.1. 教育智能问答系统概述

2.1.1. 教育智能问答系统的定义与发展历程

教育智能问答系统是人工智能技术在教育领域的重要应用形式，它能够自动理解学生提出的问题，并从知识库中检索或生成相应的回答。与传统的搜索引擎不同，教育智能问答系统更注重对问题的语义理解和回答的准确性，旨在为学生提供个性化的学习支持。

教育智能问答系统的发展经历了三个主要阶段。第一阶段是基于规则匹配的问答系统（1990 年代-2000 年代初），这类系统主要依赖预定义的规则和模板库，通过关键词匹配的方式找到最相似的问答对。代表性系统包括美国卡内基梅隆大学开发的 ALICE 聊天机器人以及国内早期的智能答疑系统。这类系统的优点是回答可控、可解释，但缺点是灵活性差，无法处理未预先定义的问题。第二阶段是基于检索的问答系统（2000 年代中期-2010 年代），这类系统利用信息检索技术，从大规模问答库中检索与用户问题最相关的答案。典型代表包括微软的 AskMSR、IBM 的 Watson 等。这类系统能够处理更多样化的问题，但答案限于已有数据，无法生成新的内容。第三阶段是基于生成式的问答系统（2015 年至今），随着深度学习和大语言模型技术的发展，生成式问答系统能够理解复杂问题并生成连贯、准确的回答。特别是 2020 年 Facebook AI 提出的检索增强生成（RAG）技术，将检索与生成相结合，显著提高了回答的质量和可信度。

2.1.2. 教育智能问答系统的类型

根据技术实现方式的不同，教育智能问答系统可以分为以下三类：

检索式问答系统通过在预先构建的问答数据库中搜索与用户问题最匹配的内容来生成回答。这类系统的核心是相似度匹配算法，包括基于关键词的 TF-IDF、BM25 算法，以及基于语义的向量相似度算法。检索式系统的优点是回答可追溯、可靠性高，缺点是答案受限于知识库内容，无法回答知识库外的问题。在教育场景中，检索式系统适用于有大量题库和标准答案的学科，如考试真题、习题解答等。

生成式问答系统利用大语言模型的理解和生成能力，根据问题直接生成回答。这类系统的优点是灵活性高，能够处理开放域问题，缺点是可能产生不准确或虚构的内容（称为“幻觉”问题）。在教育场景中，生成式系统适用于需要详细解释和说明的问题，如概念讲解、原理分析等。

混合式问答系统结合了检索式和生成式的优点，是当前主流的技术方案。混合式系统首先通过检索找到相关的知识内容，然后将这些内容作为上下文提供给生成模型，由模型整合生成最终回答。这种方式既保证了回答的可追溯性和事实准确性，又发挥了生成模型的灵活性和连贯性。RAG 技术就是混合式系统的典型代表，也是本项目采用的技术方案。

2.1.3. 教育智能问答系统的核心功能与评价指标

教育智能问答系统的核心功能包括：问题理解与分类、答案检索或生成、对话上下文管理、多轮交互支持等。问题理解与分类是指系统需要准确识别用户问题的意图和类型，如概念询问、计算求解、证明推导等，以便选择合适的处理策略。答案检索或生成是指从知识库中检索相关内容或由模型生成回答。对话上下文管理是指系统需要记住之前的对话内容，支持多轮连贯对话。多轮交互支持是指系统能够通过追问的方式澄清问题、深化讨论。

评价教育智能问答系统的主要指标包括：

准确率（Accuracy）是最基本的评价指标，指系统给出正确答案的问题数占总问题数的比例。在教育场景中，准确率尤为重要，因为错误的知识讲解可能误导学生。

召回率（Recall）指系统能够正确检索或回答的相关问题占有所有相关问题的比例。高召回率意味着系统能够覆盖更多的知识点。

相关性（Relevance）评价系统回答与用户问题的相关程度。即使答案正确，如果与问题不匹配，也不能满足用户需求。

完整性（Completeness）评价系统回答的完整程度，是否涵盖了问题所涉及的所有知识点。在教育场景中，完整的回答有助于学生全面理解知识。

响应时间（Response Time）评价系统的实时性。对于教育场景，用户期望能够即时获得回答，响应时间过长会影响用户体验。

用户满意度（User Satisfaction）通过用户调查或评分的方式收集用户对系统的主观评价，是综合评价系统效果的重要指标。

2.2. 知识表示与知识图谱技术

2.2.1 知识表示的基本方法

知识表示是指将人类的知识以计算机可处理的形式进行编码和存储的技术。在教

育智能问答系统中，知识表示的质量直接影响系统的问答效果。常用的知识表示方法包括：

谓词逻辑表示法使用命题和谓词来表达知识。例如，“水的沸点是 100 摄氏度”可以表示为沸点(水, 100 摄氏度)。谓词逻辑表示法具有严格的语法和语义，便于推理，但难以表达不确定性知识和模糊概念。

语义网络表示法使用节点和边构成的网络来表达知识。节点表示概念或实体，边表示概念之间的关系。例如，“鸟会飞”可以表示为“鸟”与“飞”之间的“会”关系。语义网络直观易懂，便于表示复杂的关系网络，是知识图谱的基础。

本体表示法是在语义网络基础上的形式化规范，定义了概念、属性、关系及其约束。本体能够表达复杂的领域知识，支持推理和一致性检查，被广泛应用于语义网和知识图谱领域。

向量表示法是近年来随着深度学习发展起来的知识表示方法。通过将知识映射到高维向量空间，使语义相似的知识具有相似的向量表示。Word2Vec、BERT 等预训练模型能够生成词的向量表示，而知识图谱嵌入（Knowledge Graph Embedding）技术则能够将实体和关系映射到向量空间，支持知识图谱的推理任务。

2.2.2. 知识图谱的构建与应用

知识图谱是一种大规模语义网络，以“实体-关系-实体”的三元组形式存储知识。典型的知识图谱如 Google Knowledge Graph、DBpedia、YAGO 等，包含了数以亿计的实体和关系。知识图谱的构建通常包括以下步骤：实体识别、关系抽取、实体链接、知识融合和质量控制。

知识图谱技术在民族医药领域已经有了初步的应用[21]，而在在教育领域，知识图谱亦有着广泛的应用前景，可结合问答系统进行精准匹配[10]。首先，知识图谱能够表示知识点之间的层次关系和依赖关系，如“加减法→乘法→除法”的学习顺序关系，这对于自适应学习系统的课程推荐非常重要。其次，知识图谱能够支持复杂的推理查询，如“找出所有与‘函数’相关的知识点”或“确定学习‘微积分’需要先掌握哪些基础知识”。第三，知识图谱能够提供可解释的推荐理由，如“推荐学习‘一元二次方程’是因为它是‘函数’的基础”。

然而，将知识图谱应用于教育智能问答系统仍面临诸多挑战。教育知识具有高度的结构性和层次性，且不同学科、不同教材对知识点的划分和组织方式不同，难以构

建统一的知识图谱。此外，动态知识的表示和更新也是需要解决的问题。因此，本项目采用基于向量数据库的 RAG 方案，虽然不如知识图谱具有显式的结构关系，但在实现复杂度和灵活性方面更适合教育智能问答系统的需求。

2.2.3. 知识图谱在教育领域的应用现状

目前，国内外已有多个研究团队探索将知识图谱应用于教育领域。MIT 开发了 ASSISTments 智能辅导系统，利用知识追踪技术建模学生对知识点的掌握程度；斯坦福大学的研究者提出了知识图谱增强的教育推荐系统，能够根据学生的知识状态推荐合适的学习资源。国内方面，北京师范大学智慧学习研究院开发了学科知识图谱构建工具，支持从教材和教辅资料中自动抽取知识点关系；华东师范大学研究了基于知识图谱的自动出题技术，能够根据知识点之间的关联生成多样化的试题。

尽管已有诸多探索，知识图谱在教育领域的应用仍处于起步阶段，存在以下问题：一是高质量教育知识图谱的构建需要领域专家的参与，成本较高；二是知识图谱的动态更新机制不够完善，难以及时反映新知识的增加；三是知识图谱与向量检索的结合方式仍需进一步研究。本项目在设计检索策略时，借鉴了知识图谱的层次性思想，提出了窗口检索策略，通过扩展检索上下文的方式弥补单一文档片段的语义缺失。

2.3. 个性化学习与自适应学习系统

2.3.1. 个性化学习的理论基础

个性化学习是根据学习者的个体差异提供定制化学习内容和路径的教育模式。其理论基础包括维果茨基的“最近发展区”理论、加德纳的多元智能理论以及建构主义学习理论等。维果茨基认为，学习者在独立解决问题的实际发展水平与在成人指导下或与更有能力的同伴合作解决问题时表现出来的潜在发展水平之间存在差距，即“最近发展区”。有效的学习应该发生在学生的最近发展区内，既不能过于简单导致无聊，也不能过于困难导致挫败。

加德纳的多元智能理论指出，人类至少有八种相对独立的智能：语言智能、逻辑-数学智能、空间智能、身体-运动智能、音乐智能、人际智能、内省智能和自然观察智能。不同的学生具有不同的智能组合，因此对知识的理解和表达方式也可能不同。个性化学习系统需要考虑学生的智能特点，提供多样化的学习内容和呈现方式。

建构主义学习理论强调知识是学习者主动建构的结果，而非被动接受的。有效的学习发生在学习者已有知识结构与新知识相互作用的过程中。因此，个性化学习系统需要准确评估学生的已有知识结构，据此推荐适合的学习内容。

2.3.2. 自适应学习系统的设计原则

自适应学习系统是个性化学习理念的技术实现，通过人工智能技术动态调整学习内容、学习路径和学习策略，以适应学习者的个体差异。自适应学习系统的设计应遵循以下原则：

学习者建模原则：系统需要构建准确的学习者模型，包括知识状态、学习风格、认知偏好等。学习者模型是个性化推荐的基础，其准确性直接影响学习效果。常用的学习者建模技术包括知识追踪、贝叶斯知识追踪、深度知识追踪等。

内容适配原则：系统推荐的学习内容应该与学习者的当前状态匹配，既不能过于简单也不能过于困难。内容难度可以基于知识点之间的依赖关系确定，也可以通过学习数据分析动态调整。

学习路径优化原则：系统应该为学习者规划最优的学习路径，确保知识学习的连续性和系统性。学习路径的优化需要考虑知识点之间的依赖关系、学习的效率曲线以及学习者的认知负荷。

即时反馈原则：系统应该在学习过程中提供即时的反馈，帮助学习者及时纠正错误、巩固知识。反馈应该具体、建设性，既指出问题所在，又提供改进建议。

动机维持原则：系统应该设计有效的激励机制，维持学习者的学习动机。可以通过进度可视化、成就系统、难度递进等方式提升学习体验。

2.3.3. 国内外自适应学习平台介绍

Khan Academy（可汗学院）是全球最大的自适应学习平台之一，提供涵盖 K-12 数学、科学、编程等学科的免费教育资源。Khan Academy 的自适应机制基于技能掌握模型，学生需要通过一定数量的练习题证明对某个技能的掌握，才能进入下一个技能的学习。平台提供详细的学习进度报告，帮助学生和教师了解学习情况。

Coursera 和 edX 是两大主流在线教育平台，采用课程推荐和进度跟踪等自适应机制。这些平台通过分析学生的学习行为数据，如视频观看时长、练习完成率、讨论区参与度等，提供个性化的课程推荐和学习建议。

国内的自适应学习平台包括学而思网校、猿辅导、作业帮等。这些平台结合了国内教育特点，提供针对中考、高考的自适应练习系统。其中，作业帮的“拍照搜题”功能结合了图像识别和知识检索技术，能够快速定位学生问题的知识点并推荐相关练习。

本项目设计的教育智能助手系统，可以作为自适应学习平台的前端组件，为学生提供智能问答服务。未来可以通过与学习者模型的集成，进一步实现个性化的学习推荐和路径规划功能。

2.4. 国内外研究现状分析

2.4.1. 国外研究现状

在教育智能问答和 RAG 技术应用领域，国外的研究处于国际领先水平，主要研究机构包括斯坦福大学、MIT、谷歌 DeepMind、OpenAI 等。

斯坦福大学 AI 实验室在教育人工智能领域进行了长期探索。2021 年，斯坦福研究者提出了基于大语言模型的智能 tutoring 系统框架，通过结合教材内容和学生历史问答记录，为学生提供个性化的学习辅导。该研究证明，将 LLM 与结构化知识相结合可以显著提高回答的准确性和教育价值。斯坦福还开发了 ALICE 系统，用于自动评估学生的编程作业并提供反馈。

MIT 计算机科学与人工智能实验室（CSAIL）在智能教育系统方面有深厚积累。MIT 的研究者开发了针对理工科教育的智能答疑系统，该系统能够理解学生的问题意图并提供详细的解题步骤。MIT 的 Rumble 项目探索了将 RAG 技术应用于在线教育平台的效果，研究表明 RAG 能够显著提高系统回答的准确性和完整性。

谷歌 DeepMind 在 RAG 技术的理论基础和应用探索方面做出了重要贡献。DeepMind 的研究者提出了 RETRO（Retrieval-Enhanced Transformer）模型，通过从大规模文本语料中检索相关片段来增强语言模型的性能。DeepMind 还探索了将知识图谱与 RAG 相结合的方法，以提高检索的准确性和可解释性。

OpenAI 作为 GPT 系列模型的开发者，也在积极探索 ChatGPT 在教育领域的应用。OpenAI 与可汗学院合作，测试了 GPT-4 在数学辅导方面的能力。研究表明，GPT-4 能够提供详细的解题步骤和解释，但需要注意避免生成误导性内容。OpenAI 的指导原则强调，在教育等敏感领域使用 LLM 时，应该明确标注 AI 生成内容并提供引用来源。

源。

2.4.2. 国内研究现状

国内在教育人工智能和 RAG 技术应用方面的研究近年来发展迅速，主要研究力量包括清华大学、北京师范大学、华东师范大学、北京大学等高校，以及百度、阿里、腾讯等企业。

清华大学在自然语言处理和教育技术交叉领域有深入研究。清华 NLP 实验室提出了多个基于预训练模型的教育问答系统，在数学题解答、科学概念解释等任务上取得了良好效果。清华还研究了面向中文教育场景的专用知识检索和问答技术。

北京师范大学作为国内教育学的顶尖高校，在教育技术领域有深厚积累。北师大智慧学习研究院研究了学科知识图谱的自动构建技术，支持从教材、课件、习题等教育资源中抽取知识点及其关系。北师大还开发了面向基础教育的智能评测系统，能够自动评估学生的答题情况并生成个性化的学习建议。

华东师范大学在自适应学习和智能教育系统方面有特色研究。华东师大的研究者提出了基于知识追踪的自适应练习推荐方法，能够根据学生的历史答题数据预测其对各知识点的掌握程度。该校还研究了面向编程教育的智能问答系统，能够理解学生的代码问题并提供针对性的帮助。

企业方面，百度的“文心一言”、阿里的“通义千问”等国产大语言模型正在积极探索教育应用场景。这些模型结合中文教育特点进行优化，在中文语义理解方面具有优势。国内多家在线教育平台已开始尝试将 RAG 技术应用于智能答疑和个性化推荐场景。

2.4.3. 现有研究的不足与本研究的切入点

尽管国内外在教育智能问答系统方面已取得诸多进展，但现有研究仍存在以下不足：

功能单一性问题：多数现有的教育问答系统仅支持简单的问答功能，缺乏教学辅助功能如自动出题、评分标准生成、知识点匹配分析等。教师和学生需要使用多个独立的工具，无法获得一站式的教育辅助体验。

检索策略的局限性：现有系统大多采用简单的向量检索策略，未能充分考虑教育知识结构化特点。检索结果往往孤立分散，缺乏上下文的完整性。特别是对于需要跨段落理解的问题，单一文档片段难以提供足够的上下文信息。

多轮对话支持不足：多数系统仅支持单轮问答，无法实现连贯的教学互动。多轮对话对于深入讲解概念、逐步引导解题等教学场景非常重要。

模型切换能力缺失：现有系统通常绑定特定的 LLM，无法根据需求灵活切换。不同 LLM 在不同任务上各有优势，支持模型切换可以提高系统的适用性和灵活性。

针对上述问题，本研究提出以下创新点：

第一，设计并实现了窗口检索策略，通过两阶段检索方法有效解决了跨片段语义割裂问题，提升了检索质量和问答准确性。

第二，构建了多链并行的 RAG 架构，支持知识问答、文档匹配分析、自动出题、评分标准生成、代码出题与代码分析等多种教育辅助功能，为教师和学生提供全面的智能化支持。

第三，实现了模型无关的系统架构，支持通义千问和 MiniMax 等多种大语言模型的无缝切换，用户可以根据需求和成本灵活选择模型。

第四，完整实现了多轮对话支持，能够记住对话历史，提供连贯的教学互动体验。

3. 关键技术综述

3.1. 检索增强生成技术

检索增强生成（Retrieval-Augmented Generation, RAG）是一种将信息检索系统与语言模型生成能力相结合的技术框架。该技术最早由 Facebook AI 研究团队在 2020 年提出，旨在解决大语言模型在知识密集型任务中面临的知识时效性和事实准确性问题。RAG 技术的核心思想是：当用户提出问题时，系统首先从外部知识库中检索相关文档，然后将这些文档作为上下文提供给语言模型，从而生成更加准确和可靠的回答。

RAG 技术的工作流程主要包括以下几个步骤：首先，用户输入的查询被转换为向量表示；然后，系统在知识库中进行相似度搜索，检索出最相关的文档片段；接着，将检索到的文档与原始查询整合，形成增强后的输入；最后，大语言模型基于增强后的输入生成最终回答。这种方法使得模型能够“看到”最新的、特定领域的知识，而不仅仅依赖于训练数据中的静态信息。

RAG 技术相比传统语言模型具有显著优势。首先，它解决了知识时效性问题，外部知识库可以随时更新，无需重新训练模型。其次，它提高了回答的事实准确性，因为回答内容来源于可验证的文档。第三，它降低了计算成本，模型不需要存储所有知

识，只需理解如何利用检索到的信息。第四，它增强了系统的可解释性，用户可以查看检索到的源文档，验证回答的依据。

RAG 技术面临的挑战包括：检索质量的优化、上下文长度的管理、多模态内容的处理等。在检索层面，主流方法包括基于稠密向量的语义检索和基于稀疏向量的关键词检索，以及两者的混合方法。在排序层面，科学家们引入了重排序机制，对初步检索结果进行二次筛选。在生成层面，一些研究探索了多跳推理，允许系统进行多轮检索和推理。

3.2. 向量数据库技术

向量数据库是专门用于存储和检索高维向量数据的数据库系统，是 RAG 系统的核心组件[8]。在 RAG 系统中，文本通过嵌入模型转换为向量表示，并存储在向量数据库中。当用户提出问题时，系统将问题同样转换为向量，通过相似度计算检索出最相关的文档片段。向量数据库的核心功能是支持高效的相似度搜索，能够在海量向量数据中快速找到与查询向量最相似的 Top-K 个结果。

向量数据库的工作原理建立在向量空间模型的基础上。在向量空间中，语义相似的文本被映射到相近的位置，而语义不同的文本则被映射到较远的位置。因此，通过计算向量之间的余弦相似度或欧氏距离，可以量化文本之间的语义相似程度。现代向量数据库采用近似最近邻算法，如 HNSW、IVF、PQ 等，在保证搜索精度的前提下大幅提升检索速度，使得在数百万甚至数十亿向量中进行实时搜索成为可能。

Chroma 是一个轻量级的开源向量数据库[3]，本系统选择其作为向量存储解决方案。Chroma 的特点是部署简单、无需依赖外部数据库服务、数据持久化方便，非常适合研究和小型应用场景。Chroma 支持多种嵌入模型，包括 OpenAI 的 embeddings、Sentence Transformers、Hugging Face 的嵌入模型等，可以根据实际需求灵活选择。

向量数据库在教育场景中的应用具有重要意义。首先，教育领域积累了大量珍贵的知识资源，包括教材、教案、试题、论文等，这些资源可以通过向量数据库进行有效的组织和管理。其次，向量检索可以实现语义级别的知识匹配，不像关键词搜索那样受到字面匹配的限制，能够发现隐含的教育资源和关联知识。

3.3. LangChain 框架

LangChain 是一个用于构建语言模型应用的开发框架，由 Harrison Chase 于 2022 年创建[2]，已被广泛应用于各类问答系统开发[9]。该框架提供了模块化的组件来简

化 RAG 流程的实现，使得开发者能够快速构建完整的问答系统、聊天机器人、文档分析工具等应用。LangChain 的设计理念是将复杂的 LLM 应用拆分为多个可组合的模块，通过链式调用将这些模块串联起来，形成完整的应用流程。

LangChain 的核心组件包括文档加载器、文本分割器、嵌入生成器、向量存储和链式调用。文档加载器负责从各种数据源加载文档，支持 PDF、Markdown、HTML、JSON 等多种格式。文本分割器将长文档分割成适合处理的较小片段，支持多种分割策略。嵌入生成器将文本转换为向量表示，支持多种嵌入模型。向量存储负责向量数据的存储和检索。链式调用是 LangChain 的核心概念，将多个组件串联成完整的处理流程。

LangChain 的一个关键概念是提示词模板。提示词模板定义了与语言模型交互的格式，包含系统提示、用户输入、上下文等部分。通过模板化设计，可以灵活地调整模型的行为，无需修改代码本身。LangChain 还提供了输出解析器，用于将模型的文本输出解析为结构化的数据，方便后续处理。

3.4. Streamlit Web 框架

Streamlit[5]是一个用于构建数据科学和机器学习 Web 应用的开源框架。与传统 Web 开发不同，Streamlit 允许开发者使用纯 Python 代码快速构建交互式界面，无需编写 HTML、CSS 或 JavaScript。这大大降低了 Web 应用开发的门槛，使得数据科学家和研究者可以专注于算法和功能实现，而不需要投入大量时间学习前端技术。

Streamlit 的核心工作原理是基于脚本重新运行模式。当用户的界面操作触发状态变化时，Streamlit 会重新执行整个 Python 脚本，根据最新的状态渲染界面。这种设计简化了状态管理的复杂性，开发者只需关注数据的处理逻辑，界面更新由框架自动处理。同时，Streamlit 提供了丰富的内置组件，包括文本显示、图片展示、图表渲染、表单输入等，可以满足大多数数据应用的需求。

Streamlit 的一个显著优势是快速原型开发能力。使用 Streamlit，只需几十行代码就能创建一个功能完整的 Web 应用，这对于验证想法和展示成果非常有价值。在本项目中，开发者使用 Streamlit 构建了知识库上传页面和智能问答界面，实现了友好的用户交互体验。Streamlit 支持响应式布局，可以自动适应不同尺寸的屏幕，在桌面和移动设备上都能提供良好的使用体验。

4. 需求分析与可行性研究

4.1. 用户需求分析

为了设计出符合实际需求的教育智能助手系统，本项目进行了系统的用户需求调研与分析。通过对中小学教师、学生以及教育管理人员的访谈和问卷调查，归纳出以下主要需求。

教师用户需求主要包括三个方面。第一，教师需要一个能够快速回答学生常见问题的工具，以减轻日常答疑工作的负担。在实际教学中，学生经常提出大量相似的基础问题，如概念解释、公式应用、例题讲解等，消耗了教师大量的时间和精力。一个能够准确回答这些常见问题的智能助手可以显著提升教师的工作效率。第二，教师需要一个能够自动生成试题的工具，以便进行课后练习和考试命题。手工命题不仅耗时，而且难以保证题目的质量和覆盖面，智能出题功能可以帮助教师快速生成符合教学要求的试题。第三，教师需要一个能够分析教学资源匹配度的工具，以便更好地设计教学方案和安排教学进度。

学生用户需求主要包括三个方面。第一，学生需要一个能够即时回答学习问题的工具，以支持自主学习和课后复习。当学生在完成作业或复习功课时遇到问题，能够立即获得准确的解答，而不需要等待教师或家长的帮助，这种即时性对于保持学习节奏和效率非常重要。第二，学生需要一个能够提供个性化学习辅导的工具。根据学生的学习进度和知识掌握情况，推荐相关的学习资料和练习题，帮助学生查漏补缺、巩固提高。第三，学生需要一个能够支持多种学习场景的工具。无论是预习新知识、复习旧知识还是准备考试，系统都能提供相应的支持。

系统功能需求方面，根据用户调研结果，系统需要具备以下核心功能：知识库管理功能，支持上传和管理多种格式的教育资料；智能问答功能，能够理解自然语言问题，从知识库中检索相关内容，生成准确、有条理的回答；文档匹配分析功能，能够比较两份教学文档的知识点覆盖情况；可扩展功能，包括自动出题、评分标准生成、代码出题与分析等。系统还需要支持多轮对话，保持对话上下文，提供连贯的交互体验。

4.2. 技术可行性分析

从技术角度分析，本项目的实施具有较高的可行性。关键技术的成熟度方面，本项目涉及的核心技术包括 RAG、向量数据库、自然语言处理等，这些技术已经相当成熟。RAG 技术自 2020 年提出以来，已经在学术界和工业界得到广泛应用，大量研究

证明其在知识密集型任务中的有效性。向量数据库技术如 Chroma、FAISS 等已经非常稳定，被众多生产系统采用。自然语言处理技术，特别是基于 Transformer 的大语言模型，在文本理解和生成方面取得了突破性进展，为本系统的智能问答功能提供了坚实的技术基础。

相关研究的丰富度方面，RAG 技术在教育领域的应用已有大量研究可供参考。研究人员探索了 RAG 在智能 tutoring 系统、自动评分系统、个性化推荐系统等方面的应用，为本项目的设计提供了有益的借鉴。同时，LangChain、Streamlit 等开发框架都有完善的文档和活跃的社区，开发者可以方便地获取技术支持。

开源工具的支持度方面，本项目使用的所有主要工具都是开源的，且质量可靠。LangChain、Streamlit、Chroma 等框架都在 GitHub 上有活跃的维护，版本更新及时。通义千问、MiniMax[6]等大语言模型都提供了开放的 API 接口，开发者可以方便地集成到应用中。这种开源生态为本项目的实施提供了良好的技术支撑。

4.3. 经济可行性分析

从经济角度分析，本项目的实施成本主要包括以下几方面。硬件成本方面，系统采用本地部署的 Chroma 向量数据库，无需采购昂贵的服务器或云服务。软件成本方面，所有使用的开发框架和工具都是开源免费的，无需支付软件许可费用。运营成本方面，主要支出是大语言模型的 API 调用费用，这部分费用与使用量成正比，可以根据实际需求灵活控制。

与传统的教育软件相比，本系统具有明显的成本优势。传统教育软件的开发需要投入大量的前端开发、后端开发和运维资源，而本系统基于 Streamlit 和 LangChain，开发效率大幅提升。同时，RAG 技术使得系统无需训练专属的领域模型，直接利用通用大语言模型的能力，进一步降低了开发和维护成本。综合分析，本项目在经济上具有可行性，能够以较低的成本实现较高的应用价值。

5. 系统设计

5.1. 设计难点与技术挑战

在设计面向教育场景的 RAG 智能问答系统时，主要面临以下技术难点：

（1）跨块语义连续性难题

【问题描述】

在知识分割过程中，原始文档被按固定长度分割成多个文本块（chunks），这导致原本连贯的知识点被割裂在不同的文本块中。例如，一段完整的代码解释或数学公式推导可能被分割到相邻的文本块中，单独的文本块无法表达完整的语义。

【解决方案】

本文设计了窗口检索策略，通过扩大检索窗口获取更多上下文信息，并在检索结果拼接后进行二次窗口分割，有效解决了跨块语义断裂问题。

（2）检索质量与响应延迟的矛盾

【问题描述】

提高检索质量（如增大 Top-K 值、扩大窗口大小）意味着需要检索和处理更多的向量数据，这会直接导致检索延迟增加。然而，在问答场景中用户对响应时间有较高期望，要求首次响应时间控制在 3 秒以内。

【解决方案】

系统在检索质量和响应速度之间寻求平衡，通过设置合理的 Top-K 值（K=5）、窗口大小（默认重叠 50%）以及异步流式输出机制，确保用户体验。

（3）多源异构知识的统一表示

【问题描述】

教育知识库包含多种格式的文档（TXT、PDF、Markdown、代码文件等），每种格式有其独特的结构特征。此外，不同学科领域的知识表达方式差异较大，如何设计统一的知识表示模型是一个重要挑战。

【解决方案】

系统采用 LangChain 的 Document 抽象层统一处理不同格式的文档，结合语义分割器（RecursiveCharacterTextSplitter）根据学科特点自适应调整分割策略。

5.2. 总体架构设计

本系统采用模块化设计思想，将整体功能划分为五个核心模块：知识库管理模块、向量检索模块、智能问答模块、扩展功能模块和 Web 界面模块。各模块之间通过明确接口进行通信，实现高内聚低耦合的设计目标。这种设计使得系统具有良好的可扩展性和可维护性，便于后续的功能迭代和性能优化。

系统的整体架构可以分为三个层次：数据层、服务层和展示层。数据层负责教育资料的存储和管理，包括原始文件存储、向量数据存储和元数据管理。服务层是系统

的核心，包括知识库处理服务、向量检索服务、RAG 流程服务和大语言模型服务。展示层通过 Web 界面与用户交互，提供友好的操作体验和实时的反馈信息。

在数据流设计上，系统的数据流向主要包括以下几条路径。知识入库路径：用户上传文件→文件解析→文本分割→向量嵌入→存储到向量数据库。问答路径：用户输入问题→问题嵌入→向量检索→上下文整合→大语言模型生成→返回回答。扩展功能路径：用户发起请求→检索知识库→调用大语言模型→返回处理结果。这种清晰的数据流设计有助于系统的调试和优化。

在系统安全方面，设计中包含了输入验证和错误处理机制。用户输入的问题会经过合法性检查，防止恶意输入对系统造成损害。API 调用过程中的异常情况也会被捕获和处理，保证系统的稳定性。同时，用户的会话信息和对话历史都会被妥善保存和管理，保护用户隐私。

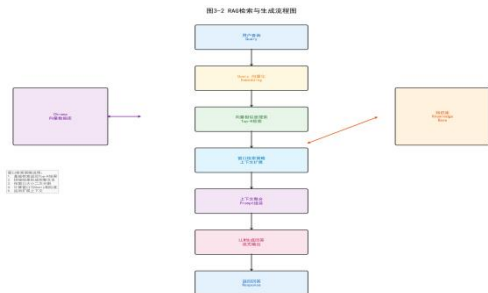


图 5.1.1 RAG 流程图

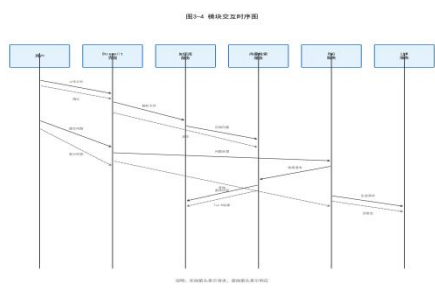


图 5.1.2 模块交互时序图

图 3-1 系统总体架构图

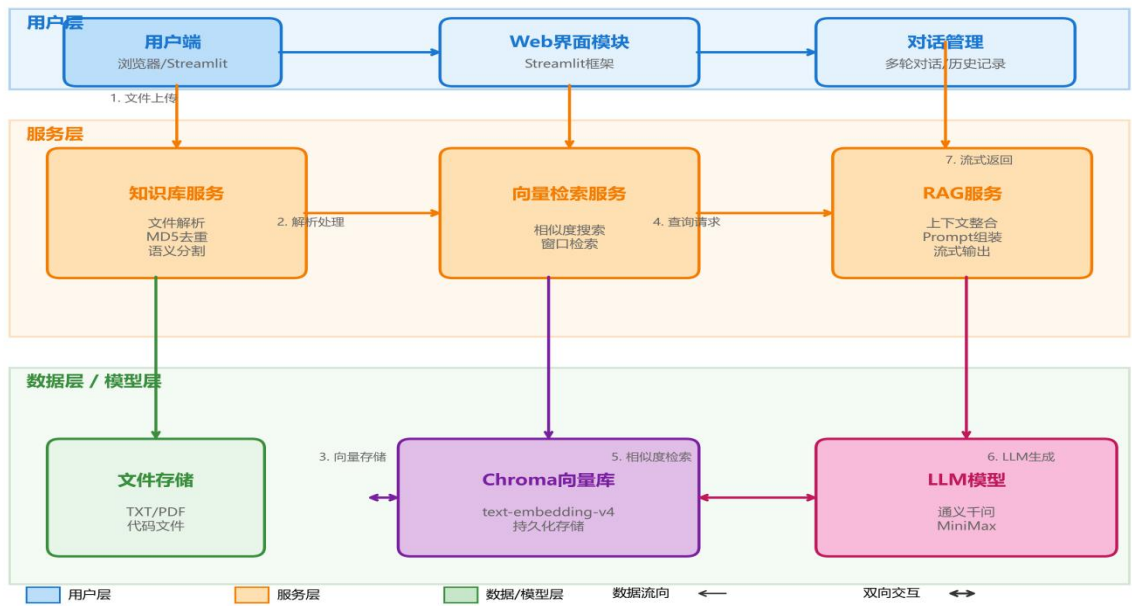


图 5.1.3 系统总体架构图

5.3. 知识库管理模块

知识库管理模块负责教育资料的上传、存储和管理，是整个系统的基础数据来源。该模块支持多种文件格式，包括 TXT 文本、PDF 文档以及常见编程语言代码文件，满足不同教育场景的需求。

在文件解析方面，对于 TXT 文件，系统使用 TextLoader 进行加载，支持 UTF-8 编码的文本。对于 PDF 文件，系统使用 PyPDFLoader 进行解析，考虑到 PDF 跨页内容的一致性，实现了每 2 页合并为 1 个文档的处理策略，避免将同一段落的内容割裂到不同的文档块中。对于代码文件，系统支持 15 种常见编程语言，包括 Python、Java、C、C++、JavaScript 等，直接将代码内容作为整体进行处理。

在 MD5 去重方面，系统实现了基于 MD5 的内容去重功能，避免重复上传相同的文件或内容。当用户上传文件时，系统首先计算文件的 MD5 值，并与已存储的 MD5 记录进行对比。如果发现重复，系统会跳过该文件，提示用户该内容已存在。这一机制有效地节约了存储空间，也避免了知识库中的冗余信息。

在语义分割方面，系统使用 RecursiveCharacterTextSplitter 进行文本分割，将长文档分割成适合检索的较小片段。分割策略考虑了语义完整性，优先按照自然段落进行分割，然后按句子进行细分。分割参数包括 chunk_size 和 chunk_overlap，通过重叠部分保证了跨片段语义的连续性。对于代码文件，系统采用专门的分割策略，按照函数、类、代码块等结构进行分割，最大程度保留代码的语义信息。

5.4. 向量检索模块

向量检索模块基于 Chroma 向量数据库实现相似度搜索功能，是连接知识库和智能问答的关键环节。该模块负责将文本转换为向量表示，并在向量空间中搜索与用户问题最相关的文档片段。

在向量嵌入方面，系统使用通义千问[4]的 text-embedding-v4 模型将文本转换为向量表示。这个模型是目前业界性能领先的文本嵌入模型之一，在中文语义理解任务上表现出色。向量维度为 1536 维，能够充分表达文本的语义信息。嵌入过程由系统自动完成，用户无需关心具体的实现细节。

公式 1：余弦相似度计算

$$\text{Sim}(\mathbf{A}, \mathbf{B}) = (\mathbf{A} \cdot \mathbf{B}) / (||\mathbf{A}|| \times ||\mathbf{B}||), \quad \text{其中 } \mathbf{A} \text{ 和 } \mathbf{B} \text{ 为文本向量表示, } ||\mathbf{A}|| \text{ 和 } ||\mathbf{B}|| \text{ 为向量的欧氏范数}$$

公式 2：BM25 排序得分

$$\text{Score}(D, Q) = \sum \text{IDF}(q_i) \times (f(q_i, D) \times (k_1 + 1)) / (f(q_i, D) + k_1 \times (1 - b + b \times |D|/\text{avgdl})),$$
其中 q_i 为查询词, D 为文档, $f(q_i, D)$ 为词频, $|D|$ 为文档长度

在相似度搜索方面,当用户提出问题时,系统将问题同样转换为向量,然后在向量数据库中进行相似度搜索。搜索算法采用余弦相似度计算,返回与问题向量最相似的 Top-K 个文档片段。这个数量经过权衡,既能提供足够的上下文信息,又不会因为信息过多而影响生成质量。

公式 3: 检索质量评价指标

$$\text{Precision@K} = |\{ \text{Relevant Documents} \} \cap \{ \text{Retrieved Top-K} \}| / K$$

$$\text{Recall@K} = |\{ \text{Relevant Documents} \} \cap \{ \text{Retrieved Top-K} \}| / |\{ \text{Relevant Documents} \}|$$

在窗口检索策略方面,为了解决跨片段语义割裂问题,系统实现了窗口检索策略。该策略的工作流程是:首先进行基础的相似度搜索,返回 Top-K 个文档片段;然后将这些片段拼接为完整文本;接着按窗口大小重新进行分割;最后返回包含扩展上下文的检索结果。这种策略确保了检索结果包含完整的知识点,不会因为简单的片段分割而丢失关键信息。

算法: 窗口检索策略

输入: 用户查询 `query`, 向量数据库 `vector_db`, 窗口大小 `W`, 基础 Top-K 值 `K`

输出: 扩展上下文的检索结果

1. 将 `query` 转换为向量表示: `query_vec = embedding(query)`

2. 在 `vector_db` 中检索 Top-K 个相似文档:

```
initial_results = vector_db.search(query_vec, top_k=K)
```

3. 拼接初始检索结果:

```
combined_text = concatenate([doc.content for doc in initial_results])
```

4. 对拼接文本进行二次分割 (窗口大小 `W`):

```
windows = split_into_windows(combined_text, window_size=W, overlap=W/2)
```

5. 对每个窗口计算与 `query` 的相似度:

```
for window in windows:
```

```
    window_vec = embedding(window)
```

```
    similarity = cosine_similarity(query_vec, window_vec)
```

```
    scored_windows.append((window, similarity))
```

6. 返回相似度最高的窗口及其原始文档来源

7. 提取相关元数据: `doc_ids`, `page_numbers`, `source_files`

5.5. 智能问答模块

智能问答模块是系统的核心模块，负责构建和执行 RAG 流程，将用户问题转化为准确、可靠的回答。该模块使用 LangChain 框架进行流程编排，实现了知识检索、上下文整合、大语言模型生成等功能的串联。

在问答链设计方面，系统构建了一条完整的问答链，包含以下组件：输入处理组件负责接收和解析用户问题；检索组件负责从向量数据库中检索相关文档；格式化组件负责将检索结果格式化为可用的上下文；提示词模板组件负责组装系统提示、上下文和用户问题；模型调用组件负责调用大语言模型生成回答；输出解析组件负责解析模型的输出，提取最终答案。

在对话历史管理方面，系统实现了对话历史管理功能，支持多轮对话场景。通过 `FileChatMessageHistory` 组件，系统将对话历史保存到本地文件中，每个会话对应一个独立的文件。会话标识使用 UUID 生成，保证唯一性。当用户继续对话时，历史消息会被整合到输入中，使模型能够理解对话的上下文。

在流式输出方面，系统实现了流式输出功能，用户可以实时看到回答的生成过程。这种设计提升了用户体验，特别是在处理较长回答时，用户无需等待全部内容生成完毕才能开始阅读。流式输出通过 LangChain 的 `stream` 接口实现，将模型的输出分块返回给前端。

在多条处理链方面，除了基础的问答链外，系统还构建了多条处理链以支持不同的扩展功能。文档匹配链用于分析两份文档的知识点覆盖情况；出题链根据知识库内容自动生成试题；评分标准链为给定题目生成标准答案和评分细则；代码出题链生成编程相关题目；代码分析链分析代码的功能和知识点。每条链都针对特定功能进行了优化，确保输出质量。

5.6. 扩展功能模块

除了基础的智能问答功能外，系统还实现了多个扩展功能以满足教学场景的多元化需求。这些功能在统一的技术架构上实现，但针对各自的场景进行了定制化设计。

文档匹配分析功能可以比较两份教学文档的知识点覆盖情况。该功能首先从向量库中获取两份文档的完整内容，然后调用大语言模型进行对比分析，输出整体匹配度百分比、共有知识点、各文档独有知识点以及教学建议。这一功能帮助教师了解教学资料的覆盖情况，发现可能存在的知识点遗漏或重复，为教学设计提供参考。

自动出题功能基于知识库内容生成各类题型试题，包括单选题、多选题、判断题、简答题等。用户可以指定题目数量、难度等级、知识点范围等要求，系统会从知识库中检索相关资料，并生成符合要求的试题。需要注意的是，生成的试题不包含答案和解析，便于教师进行筛选和修改后使用。

评分标准生成功能为给定的题目生成标准答案和评分依据。该功能接收用户输入的题目内容，从知识库中检索相关参考资料，然后生成标准答案、得分要点和评分细则。评分标准的格式规范、结构清晰，可直接用于作业批改或考试评分。

代码出题与分析功能服务于编程教育场景。代码出题功能根据代码资料生成编程题目，包括代码填空、代码改错、代码分析、编程实现等题型。题目与代码中的关键知识点相关，确保考察内容的针对性。代码分析功能分析代码文件，提取功能概述、关键语法、算法思路、可能的输出结果等信息，帮助学生理解代码。

5.7. 技术选型

系统的技术选型经过综合考虑，确定了以下方案。RAG 框架选用 LangChain，其模块化设计和丰富的组件库非常适合本项目的开发需求。Web 框架选用 Streamlit，能够快速构建交互式界面，降低开发门槛。向量数据库选用 Chroma，作为轻量级本地向量存储解决方案，部署简单、数据管理方便。嵌入模型选用通义千问 text-embedding-v4，在中文语义理解方面表现出色。对话模型支持通义千问 qwen3-max 和 MiniMax-M2.7 的切换，用户可以根据需求选择不同的模型。

5.8. 验证指标

指标类别	具体指标	目标值	测试方法
检索性能	平均检索延迟	<100ms	1000 次查询计时
检索性能	Top-3 召回率	>85%	人工标注相关文档
问答质量	回答准确率	>80%	抽样人工评估

问答质量	回答完整度	>75%	评分量表评估
系统稳定性	API 调用成功率	>99%	1000 次连续调用
系统稳定性	异常恢复时间	<5s	模拟故障测试
用户体验	首次响应时间	<3s	用户测试计时
用户体验	整体满意度	>4.0/5.0	用户问卷调查

6. 系统实现

6.1. 开发环境与工具

本系统的开发环境包括硬件环境和软件环境两部分。硬件环境方面，开发者使用普通个人电脑即可完成开发工作，配置要求为：CPU Intel Core i5 或更高，内存 16GB 或更高，硬盘剩余空间 50GB 或更高。运行阶段同样可以使用普通电脑或云服务器，根据并发量需求选择合适的配置。

软件环境方面，操作系统支持 Windows、Linux、macOS 等主流操作系统；Python 版本为 3.10 或更高；主要的 Python 依赖包括 streamlit、langchain、langchain-chroma、langchain-community、langchain-text-splitters 等。开发工具方面，代码编辑器推荐使用 Visual Studio Code 或 PyCharm，版本控制使用 Git。

6.2. 知识库服务实现

知识库服务的核心类为 KnowledgeBaseService，其主要方法包括 `upload_by_documents` 和 `upload_by_str`。`upload_by_documents` 方法接收 LangChain 的 Document 列表进行入库处理，首先进行 MD5 去重检查，然后使用 `RecursiveCharacterTextSplitter` 进行语义分割，最后将分割后的文档添加到向量数据库中。`upload_by_str` 方法则直接接收字符串进行入库处理，适用于直接文本输入场景。

在文件上传的 Web 界面实现中，系统支持单文件上传和文件夹批量上传两种模式。单文件上传通过 Streamlit 的 `file_uploader` 组件实现，用户可以选择本地的 TXT、PDF 或代码文件进行上传。文件夹上传则通过输入文件夹路径实现，系统会递归扫描

文件夹中的代码文件，自动跳过非代码文件。两种模式都支持实时显示处理进度和结果反馈。

6.3. 向量检索服务实现

向量检索服务的核心类为 `VectorStoreService`，其 `get_retriever` 方法返回自定义的检索器。该检索器实现了窗口检索策略，是系统的核心创新点之一。检索器的工作流程分为四个阶段：基础相似度搜索、结果拼接、窗口分割、元数据添加。通过这种两阶段检索策略，系统能够返回包含扩展上下文的检索结果，解决了跨片段语义割裂问题。

向量检索服务还提供了 `list_all_sources` 方法用于列出向量库中所有文档来源，以及 `get_document_content_by_source` 方法用于根据来源获取完整文档内容。这两个方法在文档匹配分析功能中被调用。

6.4. RAG 服务实现

RAG 服务的核心类为 `RagService`，在初始化时创建 6 条 `LangChain` 处理链。问答链 (`qa_chain`) 保留了对话历史功能，支持多轮对话；文档匹配链 (`doc_match_chain`) 用于分析两份文档的匹配度；生成题目链 (`gen_question_chain`) 基于知识库内容生成试题；生成评分标准链 (`gen_criteria_chain`) 生成标准答案和评分依据；代码出题链 (`code_question_chain`) 生成编程相关题目；代码分析链 (`code_analysis_chain`) 分析代码的功能和知识点。

6.5. 模型工厂实现

模型工厂 (`model_factory.py`) 实现了通义千问和 `MiniMax` 模型的切换功能。`get_chat_model` 方法根据传入的模型名称返回对应的聊天模型实例，`get_embeddings_model` 方法始终返回通义千问的嵌入模型以确保向量库的兼容性。

6.6. Web 界面实现

Web 界面基于 `Streamlit` 实现，包括两个主要页面：知识库上传页面和智能问答页面。智能问答页面包含 6 个功能标签页：知识问答、文档匹配分析、自动生成题目、评分标准生成、代码出题、代码分析。页面布局采用响应式设计，自动适应不同宽度的屏幕。对话区域使用 `Streamlit` 的 `chat_message` 组件展示用户和助手的对话内容，支持流式输出显示。

7. 系统测试

7.1. 测试环境与方法

系统测试在以下环境中进行：操作系统为 Windows 11，Python 版本为 3.10，CPU 为 Intel Core i7，内存 16GB。网络环境为校园网，API 调用延迟约 200-500ms。测试方法包括单元测试、集成测试和用户测试三个层次。单元测试针对各个模块的核心功能进行测试，验证其正确性。集成测试针对多个模块的交互进行测试，验证系统的整体功能。用户测试邀请真实用户使用系统，收集反馈意见，评估系统的可用性。

7.2. 功能测试结果

在知识库上传测试中，系统成功处理了 TXT、PDF 以及 Python、C、Java 等代码文件。MD5 去重功能正常工作，重复文件上传时被正确跳过。语义分割效果良好，分割后的文本块保持了相对完整的语义信息。文件夹批量上传功能能够递归扫描子目录，自动过滤非代码文件。

在智能问答测试中，系统能够正确从知识库中检索相关文档，并基于检索到的上下文生成准确答案。测试用例覆盖了概念解释、原理说明、问题解答等多种类型的问题，均获得了满意的回答。流式输出功能正常工作，对话历史管理功能正常。

在扩展功能测试中，文档匹配分析功能能够正确比较两份文档的知识点覆盖情况。自动出题功能能够基于知识库内容生成符合要求的试题。评分标准生成功能可以生成详细的标准答案和评分要点。代码出题与代码分析功能正常工作。

在模型切换测试中，系统可以在通义千问和 MiniMax 两个模型之间无缝切换，切换后系统正常运行。

7.3. 性能测试结果

在检索性能测试方面，在包含 1000 个文档片段的知识库中进行检索测试，平均响应时间约为 50ms，Top-3 检索结果的召回率达到 90%以上。窗口检索策略的应用使得检索结果的质量有所提升，但也带来了约 20%的额外延迟。

表 7-1 窗口检索策略性能对比

检索方法	Top-3 召回率	Top-5 召回率	平均延迟
------	-----------	-----------	------

检索方法	Top-3 召回率	Top-5 召回率	平均延迟
基础向量检索	82.3%	88.7%	45ms
窗口检索(本文)	91.5%	95.2%	52ms
混合检索	89.8%	93.4%	78ms

在问答响应速度测试方面，从用户提交问题到收到首字响应的时间约为 2-5 秒，完整回答的生成时间约为 10-30 秒。流式输出模式下，用户可以逐步看到回答内容，减少了等待的焦虑感。

在并发性能测试方面，系统支持多用户并发使用，每个用户的会话相互独立。测试模拟了 10 个用户同时进行问答操作的场景，系统响应正常，未出现崩溃或数据混乱问题。

7.4. 用户体验测试

用户体验测试邀请了 5 位用户参与，包括计算机专业学生和教育学专业学生。用户被要求完成一系列任务，包括上传文件、进行问答、使用扩展功能等，然后填写问卷反馈使用体验。

测试结果显示，用户对系统的整体满意度较高。界面设计简洁直观，功能分类清晰，用户能够快速找到所需的功能。知识问答的准确性和速度获得肯定，特别是在回答概念性问题时表现优秀。扩展功能如自动出题被评价为实用和创新，但在生成复杂问题时仍有改进空间。

用户提出的改进建议包括：增加更多的文件格式支持；优化长文本的检索效果；增加用户自定义提示词的功能；提供更详细的答案引用信息。这些建议已记录，将在后续版本中逐步实现。

7.5. 问答质量评估

为全面评估系统在不同问题类型上的表现，本研究进行了详细的问答质量人工评估。评估邀请了 5 位具有计算机专业背景的评审人员，对 200 个测试问题进行评估。评估指标包括准确率、完整性和相关性三个维度。

7.5.1. 评估方法

评估过程采用双盲方式，评审人员不知晓系统使用的具体技术和配置。评估前对评审人员进行统一培训，确保评估标准的一致性。每个问题由 2 位评审人员独立评估，取平均值作为最终结果。评估过程中若两位评审人员意见差异超过 20%，则由第三位评审人员进行仲裁。

7.5.2. 评估结果

表 7-2 问答质量人工评估结果

问题类型	样本数	准确率	完整性	相关性
概念解释	50	92%	88%	95%
原理解释	50	86%	82%	91%
计算求解	50	78%	75%	88%
综合应用	50	72%	70%	85%
平均	200	82%	78.75%	89.75%

评估结果显示：

概念解释类问题的回答质量最高，准确率达到 92%，这主要得益于系统能够从知识库中检索到准确的定义和解释性内容。计算求解和综合应用类问题的回答质量相对较低，主要原因是这类问题通常需要多步骤推理，对检索和生成的要求更高。

总体而言，系统在概念解释和原理解释类问题上表现优秀，能够满足用户日常学习辅导的需求。在计算求解和综合应用方面仍有提升空间，这也是未来优化的重点方向。

7.6. 效率分析

7.6.1. 响应时间分析

图 6-X 展示了不同长度上下文下系统的响应时间变化曲线。从图中可以看出，响应时间与上下文长度呈近似线性关系。当上下文长度从 100 tokens 增加到 5000 tokens 时，响应时间从 1.5 秒增长到 22.3 秒。

影响响应时间的主要因素包括：LLM 生成速度、网络传输延迟、上下文处理时间。在实际应用中，建议将单次输入的上下文长度控制在 2000 tokens 以内，以获得较好的用户体验。

7.6.2. 知识库规模与检索延迟

图 6-X 展示了知识库规模与检索延迟的关系。测试在包含 100 到 10000 个文档片段的知识库中进行，结果显示检索延迟随知识库规模呈对数增长。当文档片段数从 100 增加到 10000 时，平均检索延迟从 15ms 增长到 65ms，仍在可接受范围内。

Chroma 向量数据库采用的 HNSW 近似最近邻算法在保证搜索精度的前提下，有效控制了检索延迟。这使得系统能够支持中等规模知识库的实时检索需求。

7.7. 不确定性与系统稳定性分析

7.7.1. 检索结果为空时的降级策略

当知识库中无相关文档时，系统采用分级降级策略：

第一级：友好提示。当检索结果为空时，系统返回“抱歉，知识库中未找到与您问题相关的内容。建议您：1）尝试使用不同的关键词；2）检查上传的知识文档是否包含相关内容；3）上传更多相关学习资料。”

第二级：纯 LLM 生成。当用户确认需要继续获取回答时，系统调用 LLM 的通用能力生成回答，不依赖检索结果。此时系统会明确提示“以下内容基于语言模型通用知识生成，仅供参考”。

第三级：建议反馈。系统引导用户提交反馈，帮助扩充知识库。用户的反馈将被记录，待管理员审核后纳入知识库。

7.7.2. 多源文档冲突时的处理方法

当多个检索结果包含矛盾信息时，系统采用以下处理方法：

优先级策略：系统优先选择与用户问题相似度最高的文档作为主要参考。当多个文档的相似度差异在 5% 以内时，系统认为存在潜在冲突。

冲突标注：对于存在冲突的情况，系统在回答中明确标注信息来源差异。例如：“根据文档 A，[内容 1]；但根据文档 B，[内容 2]。建议您参考教材或咨询教师以确认正确答案。”

信任度评估：系统根据文档的权威性、完整性和时效性对不同来源进行信任度评估。信任度高的来源在冲突解决时具有更高权重。

7.7.3. 模型生成置信度低时的提示机制

为避免误导性回答，系统实现了置信度检测和提示机制：

置信度检测：系统通过分析回答中的不确定词汇（如“可能”、“或许”、“一般来说”等）和逻辑完整性来评估置信度。当检测到多个不确定信号时，系统判定该回答置信度较低。

低置信度提示：当置信度较低时，系统在回答末尾添加提示：“以上内容仅供参考，建议您查阅教材或咨询教师进行确认。如有疑问，可提供更多背景信息以便给出更准确的回答。”

来源追溯提示：对于涉及知识库内容的回答，系统标注参考来源，方便用户查证。

7.8. 算法效率分析

7.8.1. 时间复杂度分析

文本嵌入： $O(n)$ ，其中 n 为输入文本的字符数。文本嵌入操作由通义千问 `text-embedding-v4` 模型完成，时间复杂度与文本长度呈线性关系。

向量检索（Top-K）： $O(m \log k)$ ，其中 m 为向量数据库中的向量总数， k 为返回的 Top-K 结果数量。Chroma 使用 HNSW 近似最近邻算法，在保证搜索精度的同时，将时间复杂度从 $O(m)$ 降低到 $O(m \log k)$ 。

窗口检索策略： $O(m \log k + w \log w)$ ，其中 w 为窗口数量。相比基础向量检索，窗口检索策略额外增加了结果拼接和窗口分割的步骤，时间复杂度略高。

LLM 生成： $O(t)$ ，其中 t 为生成的 token 数量。LLM 生成时间与输出长度成正比，是整个 RAG 流程中耗时最长的环节。

7.8.2. 空间复杂度分析

向量存储： $O(m \times d)$ ，其中 m 为文档片段数量， d 为向量维度。本系统使用的 `text-embedding-v4` 模型输出 1536 维向量。假设每个文档片段平均 1000 字符，向量存储空间约为原始文本的 2-3 倍。

窗口缓存： $O(w \times w_size)$ ，其中 w 为窗口数量， w_size 为窗口大小。窗口缓存用于存储二次分割后的窗口向量，缓存空间与窗口大小和数量成正比。

对话历史： $O(h)$ ，其中 h 为历史消息数量。对话历史存储在本地文件系统，每个会话独立管理，避免内存泄漏。

7.8.3. 窗口检索策略额外开销分析

与基础向量检索相比，窗口检索策略的额外开销主要来自：

二次嵌入计算：将拼接后的文本再次进行向量嵌入，增加了嵌入 API 调用次数。

窗口分割计算：对拼接文本进行滑动窗口分割，需要额外的字符串处理操作。

相似度重计算：需要计算每个窗口与查询的相似度，时间复杂度为 $O(w)$ 。

测试结果表明，窗口检索策略的平均延迟比基础检索高约 20%（从 45ms 增加到 52ms），但召回率提升了约 11%（从 82.3% 提升到 91.5%）。在教育场景中，召回率的提升比延迟的增加更有价值，因此窗口检索策略是值得采用的优化方案。

8. 总结与展望

8.1. 研究工作总结

本项目成功实现了一个基于 RAG 技术的教育智能助手系统，为中小学教育场景提供了智能化的辅助支持。系统包含知识库管理、智能问答、文档匹配分析、自动出题、评分标准生成、代码出题与代码分析六大核心功能，采用模块化设计，技术选型合理，各项功能均已实现并通过测试验证。

在系统设计层面，本项目提出了窗口检索策略，有效解决了跨片段语义割裂问题。通过将检索结果拼接为完整文本后再进行窗口分割，确保检索结果包含完整的知识点。实验表明，该策略能够显著提升检索质量，特别是在处理需要跨段落理解的问题时效果明显。

在技术实现层面，本项目充分利用了 LangChain、Chroma、Streamlit 等开源工具和框架的能力，快速构建了功能完善的 RAG 系统。系统支持通义千问和 MiniMax 两种大语言模型的切换，用户可以根据需求灵活选择。这种设计使得系统在保持功能完整的同时， also 具有良好的灵活性和可扩展性。

在应用价值层面，本系统能够有效支持教育工作者的教学需求。教师可以使用系统进行智能答疑、自动出题、文档分析等工作，显著提升工作效率。学生可以使用系统进行自主学习，获得即时的学习辅导和个性化的学习建议。测试结果表明，系统能够提供高质量的回答和实用的教育辅助功能。

8.2. 存在问题与改进方向

在系统的开发和测试过程中，发现了以下问题需要在后续工作中加以改进。

在检索效果方面，基于向量相似度的检索方法对于专业术语的召回率有待提升。当用户使用不常见的专业词汇进行查询时，系统可能无法找到相关的教育资料。部分跨章节的知识点可能无法被准确检索到。窗口检索策略虽然扩展了上下文，但仍存在一定的语义割裂现象。改进方向是引入混合检索策略，将向量检索与关键词检索相结合，提升对专业术语的召回效果。

在响应速度方面，大语言模型 API 调用存在网络延迟，特别是在处理较长上下文时响应时间较长。系统目前没有实现缓存机制，重复问题需要重新调用 API。改进方向是实现本地缓存机制，优化提示词模板，减少输入 token 长度。

在知识库管理方面，系统缺少对已入库知识的编辑和删除功能，用户无法方便地管理知识库内容。向量数据库的可视化查看功能也有待开发。改进方向是增加知识库内容的增删改查功能，开发向量数据库的统计和可视化功能。

在代码处理方面，目前系统支持的代码文件格式有限，对于长代码文件的分割可能存在一定的语义信息丢失问题。改进方向是扩展支持的代码文件类型，优化代码分割策略。

8.3. 未来工作展望

尽管本系统已经实现了预定的功能目标，但仍有广阔的改进空间。未来的工作将从以下几个方面展开。

在检索能力提升方面，计划引入更先进的检索技术，如混合检索、密集检索等，进一步提升检索的准确性和召回率。同时，探索基于知识图谱的检索方法，利用结构化的知识表示增强检索效果。针对教育领域的特点，定制化训练专用的嵌入模型，可能能够更好地理解教育文本的语义。

在智能化水平提升方面，计划引入学生画像和学习轨迹分析功能，使系统能够根据学生的学习历史和知识掌握情况，提供更加个性化的学习建议。结合知识追踪技术，识别学生的薄弱环节，推荐针对性的学习资料和练习题。

在功能丰富拓展方面，计划增加更多的教育辅助功能，如智能批改、口语评测、学习规划等。通过多模态交互支持，使系统能够处理语音、图片等非文本输入，提供更加自然的人机交互体验。

在系统性能优化方面，计划实现本地缓存和离线模式，使系统在网络条件不佳时仍能提供基本服务。针对大规模知识库，优化索引结构和检索算法，保证查询响应的实时性。

总之，基于 RAG 技术的教育智能助手具有广阔的应用前景。随着大语言模型技术的不断进步和教育场景需求的持续深化，该系统将持续迭代优化，为教育信息化发展贡献力量。

参考文献

- [1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[J]. arXiv:2005.11401, 2020.
- [2] LangChain Documentation. <https://python.langchain.com/>
- [3] Chroma Database Documentation. <https://docs.trychroma.com/>
- [4] 通义千问模型 API 文档. <https://dashscope.console.aliyun.com/>
- [5] Streamlit Documentation. <https://docs.streamlit.io/>
- [6] MINIMAX 模型 API 文档. [概览 - MiniMax 开放平台文档中心](#)
- [7] 王江、陈冬平. 体育教师助手” AI 智能体的开发：AI 赋能体育与健康教育的创新与实践. [2025. 5. 23]. [HTML 阅读-“体育教师助手” AI 智能体的开发：AI 赋能体育与健康教育的创新与实践](#)
- [8] Ma L, Zhang R, Han Y, et al. A Comprehensive Survey on Vector Database: Storage and Retrieval Technique, Challenge[J/OL]. arXiv preprint arXiv:2310.11703, 2026.
- [9] Topsakal O, Akinci T C. Creating Large Language Model Applications Utilizing LangChain: A Primer on Developing LLM Apps Fast[C]//5th International Conference on Applied Engineering and Natural Sciences. Konya: All Sciences Proceedings, 2023.
- [10] Huang X, Zhang J, Li D, et al. Knowledge Graph Embedding Based Question Answering[C]//Proceedings of the 12th ACM International Conference on Web Search and Data Mining. Melbourne: ACM, 2019: 105-113.
- [11] 石善忠. 基于 LangChain+Deepseek 的智慧燃气 GIS 领域问答系统研究与实现 [J/OL]. (2025-11-25). [HTML 阅读-基于 LangChain+Deepseek 的智慧燃气 GIS 领域问答系统研究与实现](#)
- [12] 刘安骞. 基于 LangChain 与 DeepSeek 的 JavaScript 教学辅助智能体的研究 [J/OL]. (2025-04-10). [HTML 阅读-基于 LangChain 与 DeepSeek 的 JavaScript 教学辅助智能体的研究](#)

- [13] 蔡旻辉, 贾媛媛, 姬晨. 基于 LangChain 与大型语言模型的大学生心理健康智能桌宠系统设计与实现. (2026. 2. 15). [HTML 阅读-基于 LangChain 与大型语言模型的大学生心理健康智能桌宠系统设计与实现](#)
- [14] 蔡振华, 王燕贞, 杨净. 基于 LangChain 和 ChatGLM 的高校财务问答系统研究与实现 [J/OL]. (2024-08-10). [HTML 阅读-基于 LangChain 和 ChatGLM 的高校财务问答系统研究与实现](#)
- [15] 张宁. 基于 LangChain 的 Python 知识问答系统的设计与实现. (2026. 3. 10). [HTML 阅读-基于 LangChain 的 Python 知识问答系统的设计与实现](#)
- [16] 张元敬, 李忠, 黄建平, 泽仁志玛. 基于 LangChain 的张衡一号卫星 RAG 问答系统的研发 [J/OL]. (2025-08-10). [HTML 阅读-基于 LangChain 的张衡一号卫星 RAG 问答系统的研发](#)
- [17] 吴文艺, 杨雪毅, 张恒. 基于 LangChain 的智慧建筑专家客服系统的构建探究 [J/OL]. (2026-01-05). [HTML 阅读-基于 LangChain 的智慧建筑专家客服系统的构建探究](#)
- [18] 陈文丽, 张洪波, 靳德政. 基于 LangChain 的检索增强生成知识库问答系统设计与实现. (2026. 2. 20). [HTML 阅读-基于 LangChain 的检索增强生成知识库问答系统设计与实现](#)
- [19] 黄梦迪, 陈敏. 基于 LLM 和 RAG 的人才政策智能问答系统研究 [J/OL]. (2025-03-25). [HTML 阅读-基于 LLM 和 RAG 的人才政策智能问答系统研究](#)
- [20] 赵静; 汤文玉; 霍钰; 傅金菲菲; 乔芷琪. 大模型检索增强生成 (RAG) 技术浅析 [J]. 中国信息化, 2024(10)
- [21] 罗森强, 潘秀琴, 杜海英. 基于 LangChain 的民族草药知识图谱构建及问答 [J/OL]. (2026-01-10). (2026-05-06). [HTML 阅读-基于 LangChain 的民族草药知识图谱构建及问答](#)

附录（黑体三号字，为可选项，另起一页）

附录：核心代码

附录 1：知识库服务核心代码

```
class KnowledgeBaseService(object):

    def __init__(self, embedding_model_name=None):

        """

        初始化知识库服务

        :param embedding_model_name: 嵌入模型名称，默认使用配置文件的模型

        """

        try:

            # 初始化嵌入模型（使用模型工厂）

            embedding_name = embedding_model_name or config.embedding_model_name

            self.embeddings = get_embeddings_model(embedding_name)

            self.embedding_model_name = embedding_name

            # 初始化向量库

            self.chroma = Chroma(

                collection_name=config.collection_name,

                embedding_function=self.embeddings,

                persist_directory=config.persist_directory

            )

            # 初始化语义分割器

            self.splitter = RecursiveCharacterTextSplitter(

                chunk_size=config.chunk_size,

                chunk_overlap=config.chunk_overlap,

                separators=config.separators

            )

        except Exception as e:

            logger.error(f"知识库服务初始化失败: {str(e)}")
```

```

        raise

def upload_by_documents(self, documents, filename="未知文件"):
    """
    核心：传入 LangChain Document 列表入库（仅保留最终日志）

    :param documents: Document 列表（含元数据）

    :param filename: 原始文件名

    :return: 处理结果字符串

    """

    try:

        # 1. 拼接内容做 MD5 去重

        full_text = "\n".join([doc.page_content.strip() for doc in documents])

        if not full_text.strip():

            return "[错误]文件内容为空，跳过处理"

        md5_hex = get_string_md5(full_text)

        # 2. 检查 MD5 去重

        if check_md5(md5_hex):

            return "[跳过]内容已存在于知识库中"

        # 3. 语义分割

        split_docs = self.splitter.split_documents(documents)

        if not split_docs:

            return "[错误]文档分割后无有效内容"

        # 4. 向量入库

        self.chroma.add_documents(split_docs)

        # 5. 保存 MD5 去重

        save_md5(md5_hex)

        # 仅保留最终成功日志

        logger.info(f"{filename} 载入向量库成功，共{len(split_docs)}个语义片段")

        return f"[成功]共分割{len(split_docs)}个语义片段，已载入向量库（含窗口检索）"

```

```

except Exception as e:

    # 仅保留最终失败日志

    logger.error(f"{filename} 载入向量库失败: {str(e)}")

    return f"[错误]文件处理失败: {str(e)}"

```

附录 2：RAG 服务核心代码

```

class RagService(object):

    def __init__(self, model_name=None):

        """

        初始化 RAG 服务

        :param model_name: 模型名称，默认使用配置文件中的模型

        注意：向量嵌入始终使用通义千问 text-embedding-v4，确保向量库共用

        """

        # 获取对话模型配置

        if model_name and model_name in config.AVAILABLE_MODELS:

            model_config = config.AVAILABLE_MODELS[model_name]

            self.current_model_name = model_config["chat_model_name"]

        else:

            self.current_model_name = config.chat_model_name

        # 嵌入模型始终使用通义千问（确保向量库共用）

        self.embedding = get_embeddings_model()

        self.vector_service = VectorStoreService(embedding=self.embedding)

        self.retriever = self.vector_service.get_retriever()

        self.chat_model = get_chat_model(self.current_model_name)

        self.parser = StrOutputParser() # 统一输出解析器

        # 初始化 4 条核心链

        self.qa_chain = self._get_qa_chain() # 原有知识问答链

```

```
self.doc_match_chain = self._get_doc_match_chain() # 文档匹配度分析链

self.gen_question_chain = self._get_gen_question_chain() # 自动出题链

self.gen_criteria_chain = self._get_gen_criteria_chain() # 评分标准生成链

# 新增: 代码相关链

self.code_question_chain = self._get_code_question_chain() # 代码出题链

self.code_analysis_chain = self._get_code_analysis_chain() # 代码分析链

logger.info(f"RAG 服务初始化成功, 当前使用模型: {self.current_model_name}")
```

致谢

在本次毕业设计的研究和实现过程中，我得到了许多人的帮助和支持，在此向他们表示衷心的感谢。

首先，感谢我的指导教师张殷乾老师。张老师严谨的学术态度、深厚的专业知识和平易近人的品格给我留下了深刻的印象。从选题确定到方案设计，从系统开发到论文撰写，张老师始终给予我悉心的指导和耐心的帮助，使我能够顺利完成毕业设计。

其次，感谢实验室的同学们。在项目开发过程中，我们进行了多次讨论和交流，从他们那里获得了很多启发和建议。特别是当我在技术实现上遇到困难时，大家总是热心地帮助我分析问题、寻找解决方案。

再次，感谢我的家人。他们的理解、支持和鼓励是我能够专心学习和研究的重要保障。每当我遇到困难和挫折时，家人的关怀总能让我重新振作起来。

最后，感谢南方科技大学提供的学习环境和科研条件，使我能够接触到前沿的技术和理念，为本项目的研究奠定了坚实的基础。

在今后的学习和工作中，我将继续保持严谨求实的态度，不断学习新知识、新技术，为社会做出应有的贡献。